

História dos bancos de dados não relacionais



Apresentação

Nos últimos anos, várias aplicações viram um rápido crescimento na quantidade de conteúdo gerado pelos usuários, criando novos desafios para o armazenamento, a análise e o arquivamento de grandes quantidades de dados, muitas vezes conhecidos por *big data*. Além disso, os dados estão cada vez menos estruturados e esparsos. Esses desafios colocaram em questão a efetividade das técnicas tradicionais de gerenciamento de dados e o uso de bancos de dados relacionais, devido às suas estruturas pouco flexíveis, culminando no surgimento de uma nova classe de bancos de dados, conhecida como NoSQL. O movimento NoSQL tem como principal objetivo encontrar alternativas para problemas que não podem ser resolvidos com os bancos de dados relacionais.

Nesta Unidade de Aprendizagem, você conhecerá a origem dos bancos de dados relacionais e sua evolução até o momento atual, bem como identificará os principais bancos de dados NoSQL e como eles podem ser aplicados na prática.

Bons estudos.

Ao final desta Unidade de Aprendizagem, você deve apresentar os seguintes aprendizados:

- Descrever a história dos bancos de dados não relacionais.
- Explicar a evolução das tecnologias relacionadas a bancos de dados NoSQL.
- Identificar os principais bancos de dados NoSQL existentes, desde seu surgimento até a atualidade.



Infográfico

A história dos bancos de dados não relacionais, por incrível que pareça, começa antes mesmo da criação do modelo relacional. Ao longo dos anos, diversas tecnologias foram criadas e deram início ao que hoje se conhece como bancos de dados NoSQL.

Neste Infográfico, você verá a evolução dessas tecnologias e quais foram os precursores do movimento NoSQL.



Conteúdo do livro

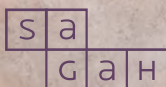
Com a globalização, a evolução tecnológica e, principalmente, a Internet, o volume de dados produzidos pela sociedade aumenta constantemente. Com isso, existe a necessidade das empresas terem capacidade de processar e armazenar grandes volumes de dados. Bancos de dados NoSQL surgiram para lidar com esse problema, quebrando alguns paradigmas definidos pelos bancos de dados relacionais.

No capítulo História dos bancos de dados não relacionais, da obra *Banco de Dados Não Relacional*, base teórica desta Unidade de Aprendizagem, você conhecerá a história dos bancos de dados NoSQL, as razões do seu surgimento e a sua evolução ao longo dos anos, bem como uma descrição detalhada de alguns dos principais bancos de dados NoSQL.

Boa leitura.

BANCO DE DADOS NÃO RELACIONAL

Maycon Bordin



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



História dos bancos de dados não relacionais

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Descrever a história dos bancos de dados não relacionais.
- Explicar a evolução das tecnologias relacionadas a bancos de dados NoSQL.
- Identificar os principais bancos de dados NoSQL existentes desde o seu surgimento até a atualidade.

Introdução

Os bancos de dados são o alicerce da maioria dos sistemas que utilizamos no nosso dia a dia. Com o avanço das tecnologias, a importância dos dados está cada vez maior, bem como a procura por formas mais eficientes de se armazenar e processar grandes volumes de dados. Assim, conhecer a história dos bancos de dados não relacionais (NoSQL) é fundamental para quem deseja trabalhar com *big data*.

Neste capítulo, você conhecerá a história dos bancos de dados não relacionais, desde o seu surgimento até os momentos atuais, com a criação e a popularização do termo NoSQL, bem como a ascensão do *big data*. Além disso, conhecerá as principais tecnologias por trás dos principais bancos de dados não relacionais. Por fim, verá quais são os bancos de dados mais relevantes. Os conceitos que serão vistos neste capítulo servirão de base para a compreensão não só da história dos bancos de dados NoSQL, mas também de como definir qual o melhor banco de dados para determinadas situações práticas, os tipos de *trade-offs* que deverão ser tomados e as consequências dessas decisões com relação a vantagens, desvantagens e limitações de cada banco de dados NoSQL.

1 Histórico

É impossível falar de bancos de dados não relacionais (NoSQL) sem antes mencionar os seus principais concorrentes, os bancos de dados relacionais. Os **bancos de dados relacionais** surgiram a partir do modelo relacional, desenvolvido por Codd em 1970 (CODD, 1970), e, até os tempos atuais, são os bancos de dados mais utilizados no mercado. Eles são conhecidos por serem sistemas de propósito geral, utilizarem a linguagem SQL (Structured Query Language; ou Linguagem de Consulta Estruturada, em português) para consulta e manipulação de dados e estruturas de dados, além das propriedades ACID para transações.

De acordo com Pritchett (2008), o acrônimo ACID reflete as características listadas a seguir.

- **Atomicidade:** as transações são atômicas, executam completamente ou não executam.
- **Consistência:** as transações criam novos estados válidos, ou, em caso de falha, o banco de dados volta para o último estado válido.
- **Isolamento:** garante que uma transação em andamento não será interferida por outras transações.
- **Durabilidade:** os dados estarão disponíveis na sua forma correta mesmo se o sistema falhar ou reiniciar.

Outra característica marcante dos bancos de dados relacionais é a integridade referencial, que garante a acurácia e a consistência dos dados dentro de um relacionamento entre tabelas. Isso é feito por meio de uma chave estrangeira, que faz referência a um valor de uma chave primária em outra tabela, e a integridade referencial garantirá que esse relacionamento é íntegro (o registro referenciado existe) (CHAPPLE, 2020).

Além disso, os bancos de dados relacionais são caracterizados pela normalização dos seus dados, um processo que busca otimizar os bancos de dados, além de garantir a maior consistência destes, evitando a duplicação de informações e as dividindo em tabelas de acordo com o tipo de elemento que está sendo armazenado (KOKAY, 2012). Desse modo, as tabelas se relacionam por meio do uso de chaves estrangeiras, que garantem a consistência das informações.

Essas características regem os bancos de dados relacionais até os dias de hoje, porém, ao longo dos anos, limitações foram sendo encontradas. Segundo Leavitt (2010), dentre as limitações de bancos de dados relacionais, as principais são: o fato de que esses sistemas não foram construídos para aplicações distribuídas; o cruzamento de dados através de *joins* tende a custar caro em termos de requisitos de *hardware* necessários; e o modelo de dados disponível é limitado, não havendo muita flexibilidade em casos em que a estrutura de dados não é completamente conhecida. Além disso, é difícil escalar esses sistemas de forma horizontal, de modo que a escalabilidade vertical (aumento de recursos de *hardware*) é a opção mais simples tecnicamente, porém é a mais custosa e, em muitos casos, pode envolver a aquisição de *hardware* proprietário.

Na metade da década de 90, com o ganho de popularidade da internet, os bancos de dados relacionais não foram capazes de acompanhar a escala das aplicações. Isso levou ao surgimento dos bancos de dados não relacionais, capazes de acompanhar a escala da internet, além de serem capazes de armazenar dados não estruturados e os processar de forma mais rápida. Os bancos de dados não relacionais também são chamados de NoSQL, termo criado em 1998 por Carlo Strozzi para se referir a um banco de dados relacional leve e *open source* que não utilizava a linguagem SQL (KUZNETSOV; POSKONIN, 2014). Contudo, o termo só ganhou a conotação que tem hoje em 2009, quando Eric Evans e Johan Oskarsson o utilizaram para se referir a bancos de dados não relacionais (STRAUCH, 2011). Ele pode ser interpretado de forma literal como “não SQL” (ausência de linguagem SQL) ou como “Not Only SQL” (Não Apenas SQL), apesar de Sadalage e Fowler (2019) recomendarem apenas o uso de NoSQL, pois até mesmo bancos de dados tradicionais, como Oracle e Postgres, poderiam se enquadrar nessa classificação.

Os bancos de dados NoSQL são caracterizados pela ausência da linguagem SQL, apesar de alguns deles apresentarem linguagens similares. Esses bancos de dados geralmente são *open source*, pois foram desenvolvidos para funcionar em *clusters* de computadores, de modo que oferecem diversas variações de consistência de dados. Além disso, eles têm como característica a ausência de esquema, permitindo a adição de novas colunas de forma livre (SADALAGE; FOWLER, 2019).

Apesar de o termo ser relativamente novo e englobar em sua classificação bancos de dados que são, em sua maioria, de código aberto, desenvolvidos no século XXI e que não utilizam a linguagem SQL, os bancos de dados não relacionais surgiram muito antes (SADALAGE; FOWLER, 2019). Os seus precursores datam da década de 60, visto que um dos primeiros bancos de dados não relacionais surgiu em 1965 na TRW, chamado de MultiValue, um banco de dados multidimensional com suporte a atributos que suporta uma lista de valores, em vez de apenas um valor (HAUGEN, 2010). Em 1979, o DBM foi lançado pela AT&T, um dos primeiros bancos de dados de chave/valor. Em 1989, surgiu o Lotus Notes, um *software* que utiliza como armazenamento interno um banco de dados orientado a documentos.

O banco de dados em grafo Neo4j surgiu no ano 2000. Em 2004, o Google BigTable foi iniciado, um banco de dados colunar distribuído, construído em cima do sistema de arquivos do Google (GFS ou Google File System). Em 2005, foi iniciado o CouchDB, outro banco de dados orientado a documentos. Em 2007, a Amazon apresentou o Dynamo, um sistema distribuído para armazenamento de dados altamente disponível que, depois, serviria de base para o Amazon S3. É importante ressaltar a relevância que a criação do BigTable (Google) e do Dynamo (Amazon) teve no mundo dos bancos de dados. De acordo com Sadalage e Fowler (2019), esses artigos levaram à criação de diversos bancos de dados NoSQL, que experimentavam formas diferentes de armazenamento de dados e passaram a ser um assunto importante nas principais conferências da área.

Em 2007, também surgiu o MongoDB, um banco de dados orientado a documentos. Em 2008, o Facebook abriu o código do banco de dados colunar Cassandra. Em 2009, surgiram dois clones *open source* do banco de dados colunar BigTable, o HBase e o Hypertable. O primeiro é *open source* e desenvolvido pela Apache, ao passo que o segundo foi criado por uma empresa que não existe mais. Dentre os bancos de dados NoSQL mais recentes, estão o banco de dados colunar Apache Druid, mais voltado para o armazenamento de dados em tempo real, e o banco de dados de grafos Apache Giraph, ambos lançados em 2011. Em 2016, foi lançada uma variante do BigTable, chamada de Apache Kudu.



Fique atento

A Google foi um dos precursores das principais tecnologias que hoje servem de base para o que se conhece como *big data*. É o caso do Hadoop e do HDFS, que foram inspirados nos artigos da Google descrevendo o MapReduce e o Google File System.

2 Evolução das tecnologias de bancos de dados NoSQL

O desafio de se armazenar e processar grandes volumes de dados com formatos diversos levou ao surgimento de bancos de dados NoSQL. Para conseguir atender a essas demandas, foi necessário abrir mão de características existentes em bancos de dados relacionais, a fim de dar lugar a outras características mais flexíveis. Isso porque lidar com grandes quantidades de dados de forma estável e permitindo que aplicações escalem a um custo baixo significa que características como a integridade garantida através de transações e a flexibilidade na criação de índices e consultas nem sempre serão possíveis (TIWARI, 2011).

Apesar de, em alguns casos, a ausência de algumas das propriedades ACID ser inaceitável, Strauch (2011) afirma que existem casos em que a redução da complexidade do banco de dados pode ser favorável, dispensando mecanismos complexos que garantiriam propriedades que nem sempre são necessárias. A perda dessas características dá lugar à capacidade de processar grandes volumes de dados em curtos espaços de tempo. Esse é o caso do Google, que é capaz de processar 20 petabytes de dados armazenados no Bigtable através do MapReduce.

Outra vantagem dessa classe de bancos de dados é que eles foram criados para escalar, ou seja, eles são capazes de responder a mais requisições e comportar volumes maiores de dados por meio da adição de um *hardware* ou da adição de novas máquinas em um *cluster*. Dessa forma, essas soluções geralmente escalam horizontalmente com maior facilidade e com um *hardware* padrão (máquinas normais com custo reduzido em relação a servidores de aplicação específicos), permitindo o *sharding* dos dados com um esforço muito menor do que o necessário para bancos de dados relacionais e sem o alto custo envolvido em bancos de dados proprietários (STRAUCH, 2011).

Além disso, os bancos de dados NoSQL, em muitos casos, tornam desnecessário o uso de mecanismos para mapeamento dos dados armazenados no banco de dados, para que eles possam se adequar a linguagens orientadas a objetos, como ORMs (Object-Relational Mapping; ou Mapeamento Objeto-Relacional, em português) para bancos de dados relacionais. Esse problema é descrito por Sadalage e Fowler (2019) como **incompatibilidade de impedância**, pois os dados possuem estruturas muito simples e dificilmente se beneficiariam das funcionalidades fornecidas por bancos relacionais e pelo SQL, ou a estrutura dos dados é muito semelhante à orientação a objetos.

Grandes empresas da internet são proeminentes no uso das tecnologias NoSQL, e a Google e a Amazon, por meio de artigos publicados descrevendo as suas alternativas aos bancos de dados relacionais, foram as propulsoras desse movimento.



Saiba mais

Sharding é uma técnica para dividir um conjunto de dados entre vários computadores de um *cluster* de banco de dados. Essa técnica permite que cada computador seja responsável por uma parcela de todos os dados, aumentando, assim, a capacidade do banco de dados de responder a requisições e de armazenar dados.

Hoje, o uso de bancos de dados NoSQL é largamente difundido, principalmente em empresas de tecnologia, devido, em parte, ao baixo custo de implementação e à alta escalabilidade. Em contrapartida, tecnologias mais tradicionais de bancos de dados de alto desempenho (p. ex., Oracle RAC [Real Application Cluster]) são utilizadas apenas por empresas com grande poder aquisitivo e que geralmente estão presas a tecnologias proprietárias ou não podem se arriscar a utilizar bancos de dados NoSQL.

A seguir, serão descritas algumas das técnicas e dos conceitos que permitem a criação e a evolução dos bancos de dados NoSQL.

Teorema CAP

Em sistemas distribuídos, observaram-se algumas características desejáveis, bem como algumas restrições com relação a essas características e a necessidade de se fazer um *trade-off* entre elas. Introduzido por Eric Brewer, esse *trade-off* é chamado de **teorema CAP** (GILBERT; LYNCH, 2012). A seguir, são listados os significados do acrônimo CAP (SADALAGE; FOWLER, 2019).

- **Consistency (consistência):** indica se e quando um sistema está em um estado consistente após a execução de uma operação. Por exemplo, se uma atualização é feita em um escritor do sistema e todos os leitores do sistema conseguem ver essa atualização imediatamente, o sistema é consistente.
- **Availability (disponibilidade):** o sistema deve permanecer disponível mesmo após a falha de nós ou a indisponibilidade de algum *hardware* ou *software*.
- **Partition tolerance (tolerância a falhas de partição):** habilidade de um sistema de se manter operante mesmo com partições na rede, isolando alguns nós em ilhas. Pode significar, ainda, a capacidade de um sistema de lidar com a adição ou a remoção de nós.

Segundo o teorema CAP, um sistema distribuído só pode ter duas das três propriedades descritas, gerando, assim, três combinações possíveis de sistemas (ZVAREVASHE; GOTORA, 2014), descritas a seguir.

1. **CA:** são sistemas que garantem a consistência dos dados e possuem alta disponibilidade, mas não suportam falhas de partição. Por exemplo: Vertica, Greenplum.
2. **CP:** são sistemas que garantem a consistência dos dados e são tolerantes a falhas de partição, porém são bancos de dados que não têm boa disponibilidade. Esses sistemas possuem as propriedades ACID. Por exemplo: Google BigTable, Hypertable, HBase, MongoDB.
3. **AP:** são sistemas que estão sempre disponíveis, mas podem apresentar inconsistências nos dados. Esses sistemas possuem as propriedades BASE. Por exemplo: Voldemort, Tokyo Cabinet, CouchDB, Riak.

Consistência eventual

Com o advento de sistemas largamente distribuídos, garantias fortes de consistência passaram a ser deixadas de lado, dando espaço para modelos mais fracos, sendo a consistência eventual o modelo mais notável (BAILIS; GHODSI, 2013). A consistência eventual fornece algumas garantias: caso não haja mais nenhuma atualização em determinado dado, todas as leituras para esse dado, eventualmente, retornarão o mesmo valor. Esse modelo não descarta a possibilidade de o cliente ler um dado inconsistente, mas garante que, em algum momento do futuro, eventualmente o dado estará consistente. Uma das vantagens da consistência eventual é a possibilidade de criar sistemas de larga escala capazes de continuar operando mesmo na presença de falhas. Ou seja, eles estão sempre disponíveis, mesmo que isso signifique que uma parcela dos usuários verá dados desatualizados por um determinado período de tempo.

Segundo Bailis e Ghodsi (2013), os sistemas eventualmente consistentes geralmente propagam atualizações de forma assíncrona, garantindo que, em algum momento, todo o sistema convergirá para um estado consistente, ou seja, todos os nós terão a mesma cópia do dado atualizada. A propagação de atualizações, que podem ocorrer de forma concorrente para um mesmo dado, implica o uso de mecanismos para garantir que os nós de um sistema consigam saber qual é a atualização mais nova que deve ser aplicada. Um desses mecanismos é o uso de um valor de relógio atrelado a cada escrita, permitindo, assim, o uso de uma regra simples, como a *last write wins* (a última escrita ganha). Entretanto, esse mecanismo requer que os relógios de todos os nós estejam sincronizados para que as escritas sejam aplicadas corretamente; isso pode ser alcançado (CORBETT *et al.*, 2013) ou contornado por outras técnicas, como é o caso dos Vector Clocks.

Outro problema que pode surgir com a consistência eventual é que, se um cliente efetua uma atualização, dependendo do sistema, existe uma chance de que ele não veja as suas atualizações instantaneamente. Isso pode gerar alguns problemas, pois o cliente pode pensar que a sua atualização não ocorreu e a enviar novamente. Contudo, esse problema pode ser resolvido através do princípio RYOW (Read Your Own Writes; ou Leia as suas Próprias Escritas, em português), que afirma que os clientes devem ver as suas atualizações imediatamente, mesmo que a atualização ocorra em um servidor, e a leitura, em outro (FAOUR, 2018). Outra forma de se conseguir isso é através da consistência de sessão, ou seja, um cliente é associado a um único servidor, garantindo, assim, que suas atualizações estejam visíveis a ele.

Vector Clocks

Segundo Prakash e Singhal (1997), o Vector Clocks é um mecanismo para controlar a dependência entre eventos causais de processos diferentes, os quais não dependem de relógios sincronizados entre os nós, não necessitam de uma ordem total de revisões para causar *reasoning* e não precisam armazenar e manter várias revisões de um dado em vários nós. Os Vector Clocks podem ser utilizados para a propagação de estado entre os nós através de protocolos Gossip (fofoca) ou epidêmicos, este podendo operar tanto pelo modelo de transferência de estado (State Transfer Model) como pelo de operações (Operation Transfer Model).

Os protocolos Gossip conseguem entregar apenas consistência eventual, visto que um *cluster* com n nós irá demorar $O(\log n)$ rodadas para propagar as suas atualizações. Em contrapartida, esse método fornece escalabilidade ao sistema e evita pontos únicos de falha (SPoF, Single Point of Failure). De acordo com Shapiro *et al.* (2011), além dos protocolos Gossip, existem outros métodos para propagação de atualizações para réplicas, como o método de antientropia.

A consistência eventual do protocolo Gossip levanta ainda outras questões, já que um sistema pode ter nós em estados diferentes ao mesmo tempo, exigindo algum tipo de versionamento dos dados e alguma forma de comunicar as atualizações para outros nós.

Sharding

A escalabilidade de bancos de dados NoSQL segue a estratégia de escalabilidade horizontal para passar pela clusterização de servidores de banco de dados, o uso de réplicas e o *sharding* dos dados. A utilização de *cache* em memória também pode ser adotada como estratégia para o armazenamento de dados que podem estar distribuídos, removendo parte da carga do banco de dados principal. Contudo, essa solução também apresenta limites de escalabilidade.

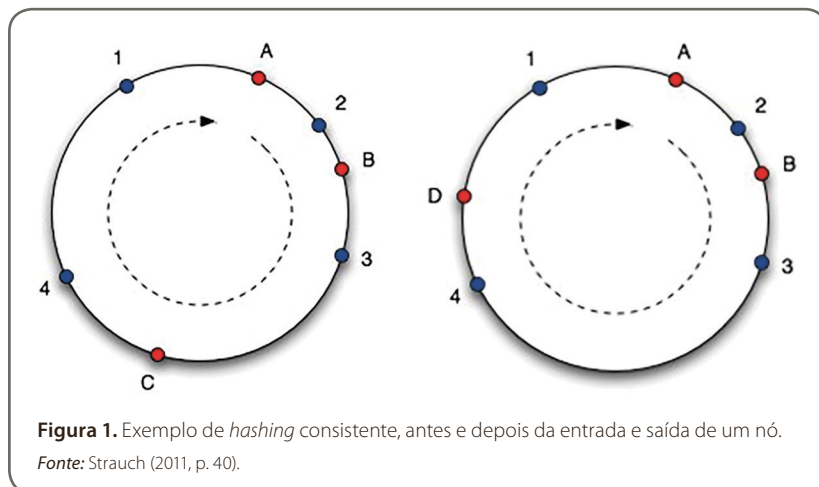
As técnicas listadas anteriormente também são utilizadas em bancos de dados relacionais, e, nos bancos de dados NoSQL, existem desafios com relação ao *sharding* de dados, principalmente no que diz respeito ao mapeamento dos dados em servidores (STRAUCH, 2011). Essa função é feita normalmente através do *hashing*, fazendo uso, por exemplo, da chave primária dos objetos para decidir em qual servidor o dado será armazenado:

$$\text{partition} = \text{hash}(o) \bmod n \text{ with } o = \text{object to hash}, n = \text{number of nodes}$$

A fórmula apresentada é um exemplo de como a escolha da partição onde o objeto será armazenado pode ser realizada. Nesse caso, é gerada uma *hash* a partir do objeto a ser armazenado e efetuada a operação de módulo utilizando o número de nós disponíveis no *cluster*. Entretanto, essa solução apresenta um problema, pois ela não leva em consideração que os nós do *cluster* podem aparecer e desaparecer a qualquer momento (falha no servidor, na rede, no banco de dados), tornando alguns dados indisponíveis.

Para resolver esse problema, pode-se utilizar a técnica do *hash* consistente, em que cada nó do sistema recebe da função de *hashing* um intervalo determinado, e todos os objetos que se enquadrarem nesse intervalo serão ali armazenados. Esses intervalos são muito flexíveis, de modo que, se um nó sai do sistema, outros nós tomarão conta do seu intervalo, da mesma forma que, se um novo nó se junta ao sistema, ele receberá parte do intervalo de outro nó. Isso permite que os clientes calculem em que nó determinado dado está sem a necessidade de um servidor de metadados, como ocorre no Google File System (GFS).

A Figura 1, a seguir, apresenta um exemplo de *hashing* consistente, em que o anel à esquerda representa o estado inicial de um sistema com três nós (A, B e C) e quatro objetos (1, 2, 3, 4). O anel funciona em sentido horário, ou seja, cada objeto no anel está armazenado no nó à sua esquerda. No anel à direita, o nó C saiu por algum motivo, ao passo que o anel D entrou no sistema. Este último tomou metade dos objetos do nó A (um) e passou a cuidar dos objetos do nó C.



Sem muito esforço, é possível observar a falta de balanceamento na distribuição dos objetos entre os nós. Para superar isso, foi introduzido o conceito de **nós virtuais**, em que cada nó real pode ter um ou mais nós virtuais, de acordo com a sua capacidade de *hardware*, e são os nós virtuais que receberão os intervalos de *hashing*. Ainda assim, o problema de indisponibilidade de dados no caso da saída de um nó persiste. Para resolver esse problema, foi introduzido o fator de replicação (r), que determina que não apenas um nó físico fique responsável por um objeto, mas r nós físicos.

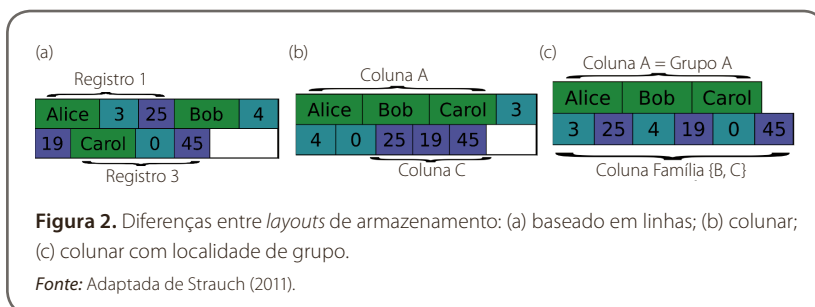
A natureza dinâmica dos nós em um sistema distribuído só pode ser alcançada por meio da comunicação entre eles. Cada vez que um novo nó se junta ao sistema, ele precisa anunciar a sua chegada aos demais nós (ou adjacentes), para que estes ajustem as suas propriedades de dados, cedendo parte para o nó recém-chegado, que pode os copiar de só uma vez ou de forma assíncrona. Já quando um nó sai do sistema, geralmente ele não dá um aviso prévio (dependendo da situação que levou à sua saída), deixando a responsabilidade de perceber a saída de um nó para os demais, que devem se comunicar entre si periodicamente para perceber tais eventos. Uma vez notada a saída de um nó, os demais precisam ajustar as suas propriedades de objetos, trocando dados entre si.

Armazenamento de dados

Em se tratando da forma como os dados são armazenados, os bancos de dados NoSQL apresentam diferenças com relação aos bancos de dados relacionais. Essas diferenças referem-se à forma como os dados são armazenados e acessados em disco e aos modelos de consultas utilizados. Com relação ao primeiro aspecto, ele determina como os dados serão acessados em disco, afetando diretamente o desempenho, além de indicar que tipo de dados poderá ser lido em blocos. Os principais *layouts* para armazenamento são descritos a seguir (TIWARI, 2011).

- **Baseado em linhas:** nesse *layout* (Figura 2a), uma tabela é serializada com linhas inteiras, adicionadas ao fim da tabela, facilitando o acesso a conjuntos de dados em uma única operação de IO, pois possuem boa localidade de acesso a diferentes colunas.
- **Colunar:** ao contrário do *layout* baseado em linhas, no colunar (Figura 2b), as tabelas são serializadas com colunas, em vez de linhas, beneficiando operações sobre colunas, muito úteis em análise de dados, mas a custo de um desempenho sofrível em operações sobre linhas.

- **Colunar com grupos de localidade:** semelhante ao *layout* colunar, porém introduz os grupos de localidade, permitindo que as colunas sejam agrupadas quando se espera que elas sejam acessadas ao mesmo tempo (Figura 2c).
- **Árvores LSM (Log Sctructured Merge):** esse *layout* contrasta com todos os outros descritos, pois ele não procura serializar estruturas de dados lógicas, mas sim utilizar a memória e o disco de forma eficiente, servindo tanto requisições de leitura como escrita de forma eficiente, com alto desempenho e de forma segura. Os dados são armazenados em pedaços na memória (Memtables) e *commit-logs* são mantidos em disco para esses dados. De tempos em tempos, os dados em memória são descarregados em disco em SSTables.



Já com relação às funcionalidades de consulta em bancos de dados NoSQL, elas são muito variadas, partindo da premissa de simples *lookups* de chaves primárias, como é o caso dos bancos de dados de chave/valor. Além disso, existem algumas técnicas, como o *scatter/gather*, em que uma consulta é enviada para vários nós do banco de dados e, posteriormente, os resultados retornados são processados e entregues ao cliente. Um exemplo disso é o MapReduce, introduzido pela Google em 2004, em que pedaços de dados são designados a determinados nós que irão executar uma função *map* e devolver um resultado, o qual será processado por máquinas através da função *reduce* para criar o resultado final.



Saiba mais

O MapReduce é um modelo de processamento de dados muito simples, o qual deu início ao movimento *big data*. Hoje, existem diversos modelos e *frameworks* para processamento de grandes volumes de dados, como o Hadoop e o Spark.

3 Principais bancos de dados NoSQL

Ao longo dos anos, diversos bancos de dados surgiram no mercado e ganharam proeminência. A seguir, serão apresentados os bancos de dados NoSQL mais utilizados no mercado de acordo com o *site* db-engines.com.

DBM

Criado por Ken Thompson e lançado pela AT&T em 1979, o DBM foi um dos precursores dos bancos de dados de chave/valor. Ele consiste em uma biblioteca para armazenamento de dados com um número fixo de *buckets* e técnicas de *hashing* para a recuperação rápida de dados (BRACHMAN; NEUFELD, 1992). Ao longo dos anos, o DBM foi reescrito diversas vezes, sendo que, em 1991, foi reescrito pela então SleepyCat Software e chamado de Berkeley DB, hoje um produto da Oracle.

Os bancos de dados de chave/valor Tokyo Cabinet e Kyoto Cabinet são implementações inspiradas no DBM, bem como o WiredTiger, um banco de dados com armazenamento orientado tanto a colunas como a linhas e que, hoje, é a *engine* por baixo do MongoDB (FEDOROVA *et al.*, 2018).

Neo4j

O Neo4j é um banco de dados orientado a grafos *open source* escrito em Java, com suporte a transações ACID, acessível através da linguagem CQL (Cypher Query Language; ou Linguagem de Consulta Contextual, em português). A comunicação com o banco de dados pode ser feita através do protocolo HTTP (HyperText Transfer Protocol; ou Protocolo de Transferência de Hipertexto, em português) ou do protocolo Bolt. Esse banco de dados possui uma versão *community* livre para uso e outra versão *enterprise* paga, que fornece funcionalidades como *backup* e alta disponibilidade.

O Neo4j é utilizado para sistemas de recomendações, aplicações da Internet das Coisas (IoT, Internet of Things) em análise de redes sociais, para traçar a árvore genealógica, para catálogo de produtos e até mesmo para buscas na internet.

Google BigTable

Desenvolvido pela Google em 2004, o BigTable é um banco de dados colunar com compressão e de alta *performance* (CHANG *et al.*, 2008). Ele depende do GFS para o armazenamento dos dados no formato de SSTables, além de utilizar o paradigma de programação Mapreduce, também de autoria da Google, para a geração e a modificação de dados armazenados no BigTable.

CouchDB

Banco de dados orientado a documentos criado em 2005 e escrito em Erlang. Os documentos são armazenados no formato JSON, e as consultas, na forma de Mapreduce, e consultas via API HTTP podem ser realizadas. O CouchDB tem suporte a propriedades ACID no nível de documento, consistência eventual e replicação de dados (ANDERSON; LEHNARDT; SLATER, 2010).

Amazon Dynamo

Criado pela Amazon em 2007, o Dynamo é um banco de dados de chave/valor altamente disponível. Ele abre mão da garantia de isolamento dentre as propriedades ACID para garantir uma alta disponibilidade. O Dynamo foi utilizado como base para vários serviços da AWS, como o Amazon S3 e o DynamoDB (DECANDIA *et al.*, 2007).

De acordo com Decandia *et al.* (2007), o Dynamo faz uso de *hashing* consistente para o particionamento de dados, permitindo a escalabilidade linear do *cluster*. Além disso, ele utiliza Vector Clocks para a reconciliação de atualizações durante a leitura, bem como uma técnica de antientropia com Merkle tree para sincronizar réplicas com divergências e um protocolo de filiação baseado no Gossip para evitar ter um componente centralizado para controlar os nós do *cluster*.

MongoDB

O MongoDB é um banco de dados orientado a documentos que utiliza o formato BSON (Binary JSON) para armazenar os documentos. Ele foi escrito em C++, possui suporte completo a índices, realiza o mapeamento de arquivos em memória (com escrita atrasada) e possui suporte a alta disponibilidade e escalabilidade através de replicação, *failover* e *auto-sharding* (HOWS; MEMBREY; PLUGGE, 2015).

O MongoDB é um dos bancos de dados NoSQL mais populares e vem sendo utilizado para substituir os bancos de dados relacionais em aplicações *web*. Além disso, ele é utilizado no gerenciamento de conteúdo semiestruturado, na análise de dados em tempo real, para *log* de dados em alta velocidade e para *cache* e alta disponibilidade de dados. O MongoDB possui, ainda, suporte para transações, garantindo a atomicidade de operações dentro de um documento (o suporte a transações, abrangendo múltiplos documentos, passou a ser suportado na versão 4.0). Entretanto, ele não é recomendado para aplicações altamente transacionais e problemas que necessitem de SQL.

Cassandra

O Cassandra é um banco de dados colunar e distribuído, descentralizado e *open source* capaz de gerenciar grandes volumes de dados com alta disponibilidade e sem SPoF. Ele é capaz de escalar de forma linear até mais de mil nós, com uma arquitetura *peer-to-peer*, suporte *multi-data center* e uma linguagem de consulta chamada de CQL, similar à linguagem SQL (CARPENTER; HEWITT, 2016).

Dentro dos conceitos do Cassandra, o *keyspace* é equivalente a um *data-base* em bancos de dados relacionais, uma *column family* é equivalente a uma tabela, uma *row key* é equivalente a uma chave primária e colunas e valores são iguais em ambos os paradigmas. De acordo com Carpenter e Hewitt (2016), no Cassandra, a busca de registros pela chave é muito rápida, e os dados são armazenados de forma ordenada, permitindo buscas por *ranges* de valores. Contudo, o número de colunas permitido no Cassandra é ilimitado, e os nomes de colunas podem chegar a 64 KB, ao passo que os seus valores podem chegar até 2 GB.

Redis

O Redis (REmote DIctionary Service) é um banco de dados de chave/valor *open source* escrito em C com armazenamento em memória com persistência em disco. Ele pode ser utilizado como banco de dados, camada de *cache* ou sistema de mensageria.

O Redis é um banco de dados compatível com sistemas POSIX (Linux, BSD, OS X), possui um servidor *single-threaded*, em que todas operações são atômicas, e a maioria dos seus comandos executa com complexidade $O(1)$. Todos os dados são escritos em memória, o que garante uma leitura e escrita muito rápidas. Além disso, os dados podem ser salvos em disco. O Redis também tem suporte à replicação e à divisão de dados através de *sharding* (CARLSON, 2013). Essas características fazem do Redis um banco de dados muito popular, sendo utilizado por empresas como Hulu, Twitter, Pinterest, Snapchat, GitHub, Airbnb e Square, além do fato de possuir clientes para mais de 50 linguagens de programação.

Apache HBase e Hypertable

O Hbase é um banco de dados colunar baseado no Google Bigtable, escrito em Java e com dependência no HDFS (Hadoop Distributed File System). O Hypertable também é um banco de dados colunar baseado no Google Bigtable, mas, ao contrário do HBase, ele foi escrito em C++ e não é *open source* (KHETRAPAL; GANESH, 2008).

Apache Druid

O Apache Druid é um banco de dados colunar *open source* escrito para ter alto desempenho na escrita de grandes volumes de dados em tempo real, além de permitir a indexação de dados para consultas em tempo real (YANG *et al.*, 2014).

Apache Kudu

O Apache Kudu é um banco de dados colunar *open source* compatível com o ambiente Hadoop. Ele foi desenvolvido para o processamento de dados analíticos, além de ter suporte para índices, permitindo o *lookup* aleatório de dados. O modelo de dados do Apache Kudu é muito similar ao de um banco

de dados relacional, entretanto, a sua arquitetura permite a escalabilidade dos dados (BARANOWSKI *et al.*, 2019).

ScyllaDB

O ScyllaDB é um banco de dados colunar *open source* com compatibilidade com o Apache Cassandra por meio do suporte dos mesmos protocolos utilizados pelo Cassandra: Thrift e CQL. Em contrapartida, o ScyllaDB oferece um *throughput* maior e latências menores do que o Cassandra, utilizando o mesmo formato de arquivos (SSTable), mas sendo escrito em C++, em vez de Java (SUNEJA, 2019).

O Quadro 1, a seguir, apresenta os principais bancos de dados NoSQL disponíveis.

Quadro 1. Lista dos principais bancos de dados NoSQL

Banco de dados	Licença	Tipo	Ano de criação	Criado/Mantido por
MultiValue (PICK)	Proprietário	Multivalor	1965	TRW
DBM	Proprietário	Chave/valor	1979	AT&T
Lotus Domino	Proprietário	Documentos	1989	Lotus
GDBM	<i>Open source</i>	Chave/valor	1990	GNU
BerkeleyDB	<i>Open source</i>	Chave/valor	1991	Sleepycat/Oracle
TDBM	<i>Open source</i>	Chave/valor	1992	
Caché	Proprietário	Multivalor	1997	InterSystems
Metakit	<i>Open source</i>	Documentos	1997	Jean-Claude Wippler
Neo4j	<i>Open source</i>	Grafos	2000	Neo4j
db4o	<i>Open source</i>	Objetos	2000	Action
Memcached	<i>Open source</i>	Chave/valor	2003	Danga Interactive

(Continua)

(Continuação)

Quadro 1. Lista dos principais bancos de dados NoSQL

Banco de dados	Licença	Tipo	Ano de criação	Criado/ Mantido por
BigTable	Proprietário	Colunar	2004	Google
Infogrid	<i>Open source</i>	Grafos	2005	
CouchDB	<i>Open source</i>	Documentos	2005	Apache
Tokyo Cabinet	<i>Open source</i>	Chave/valor	2006	FAL Labs
Dynamo	Proprietário	Chave/valor	2007	Amazon
MongoDB	<i>Open source</i>	Documentos	2007	MongoDB Inc.
Cassandra	<i>Open source</i>	Colunar	2008	Facebook/ Apache
Project Voldemort	<i>Open source</i>	Chave/valor	2008	LinkedIn/ Microsoft
Dynomite	<i>Open source</i>	Chave/valor	2008	
Hypertable	<i>Open source</i>	Colunar	2008	Zevents Inc.
Accumulo	<i>Open source</i>	Colunar	2008	Apache
Terrastore	<i>Open source</i>	Documentos	2009	Sergio Bossa
Redis	<i>Open source</i>	Chave/valor	2009	Redis Labs
Riak	<i>Open source</i>	Chave/valor	2009	Basho Technologies
HBase	<i>Open source</i>	Colunar	2009	Apache
Vertexdb	<i>Open source</i>	Grafos	2009	
ClickHouse	<i>Open source</i>	Colunar	2009	Yandex
RethinkDB	<i>Open source</i>	Documentos	2009	
OrientDB	<i>Open source</i>	Multimodelo	2010	OrientDB Ltd.
Couchbase	<i>Open source</i>	Documentos	2010	Couchbase Inc.
CosmosDB	Proprietário	Documentos	2010	Microsoft
LDBM	<i>Open source</i>	Chave/valor	2011	

(Continua)

(Continuação)

Quadro 1. Lista dos principais bancos de dados NoSQL

Banco de dados	Licença	Tipo	Ano de criação	Criado/Mantido por
ArangoDB	<i>Open source</i>	Multi-modelo	2011	ArangoDB GmbH
Druid	<i>Open source</i>	Colunar	2011	Apache
DynamoDB	Proprietário	Chave/valor, documentos	2012	Amazon
FoundationDB	<i>Open source</i>	Multimodelo	2013	Apple
Scylla	<i>Open source</i>	Colunar	2014	ScyllaDB Inc.
Kudu	<i>Open source</i>	Colunar	2016	Cloudera/ Apache
TerminusDB	<i>Open source</i>	Grafos	2018	
Neptune	Proprietário	Grafos	2018	Amazon
DocumentDB	Proprietário	Documentos	2019	Amazon



Referências

ANDERSON, J. C.; LEHNARDT, J.; SLATER, N. *CouchDB: the definitive guide: time to relax*. Sebastopol: O'Reilly, 2010.

BAILIS, P.; GHODSI, A. Eventual consistency today: limitations, extensions, and beyond. *ACM Queue*, New York, v. 11, n. 3, p. 20-32, 2013.

BARANOWSKI, Z. *et al.* A prototype for the evolution of ATLAS EventIndex based on Apache Kudu storage. In: INTERNATIONAL CONFERENCE ON COMPUTING IN HIGH ENERGY AND NUCLEAR PHYSICS, 23., 2018, Sofia, Bulgaria. *Anais eletrônicos...* Sofia: EPJ Web of Conferences, 2019.

BRACHMAN, B. J.; NEUFELD, G. W. TDBM: a DBM library with atomic transactions. In: USENIX SUMMER 1992 TECHNICAL CONFERENCE, 1992, San Antonio. *Anais eletrônicos...* San Antonio: USENIX, 1992. p. 63-75.

CARLSON, J. *Redis in action*. Shelter Island: Manning, 2013.

CARPENTER, J.; HEWITT, E. *Cassandra: the definitive guide: distributed data at web scale*. 2. ed. Sebastopol: O'Reilly, 2016.

CHANG, F. *et al.* Bigtable: a distributed storage system for structured data. *ACM Transactions on Computer Systems (TOCS)*, New York, v. 26, n. 2, p. 1-26, 2008.

CHAPPLE, M. How referential integrity ensures database consistency. *In: LIFEWIRE*. New York, 5 apr. 2020. Disponível em: <https://www.lifewire.com/referential-integrity-definition-1019181>. Acesso em: 27 jun. 2020.

CODD, E. F. A relational model of data for large shared data banks. *Communications of the ACM*, New York, v. 13, n. 6, p. 377-387, jun. 1970.

CORBETT, J. C. *et al.* Spanner: Google's globally distributed database. *ACM Transactions on Computer Systems (TOCS)*, New York, v. 31, n. 3, p. 1-22, 2013.

DECANDIA, G. *et al.* Dynamo: Amazon's highly available key-value store. *ACM SIGOPS operating systems review*, New York, v. 41, n. 6, p. 205-220, 2007.

FAOUR, N. Data consistency simulation tool for NoSQL database systems. *In: arXiv*. Ithaca, 2018. Disponível em: <https://arxiv.org/abs/1802.08052>. Acesso em: 27 jun. 2020.

FEDOROVA, A. *et al.* Performance comprehension at WiredTiger. *In: ACM JOINT MEETING ON EUROPEAN SOFTWARE ENGINEERING CONFERENCE AND SYMPOSIUM ON THE FOUNDATIONS OF SOFTWARE ENGINEERING*, 26., 2018, Lake Buena Vista. *Anais...* New York: Association for Computing Machinery, 2018. p. 83-94.

GILBERT, S.; LYNCH, N. Perspectives on the CAP theorem. *Computer*, New York, v. 45, n. 2, p. 30-36, 2012.

HAUGEN, K. A brief history of NoSQL. *In: ALL about the code. [S. l.]*, 16 mar. 2010. Disponível em: <http://blog.knuthaugen.no/2010/03/a-brief-history-of-nosql.html>. Acesso em: 27 jun. 2020.

HOWS, D.; MEMBREY, P.; PLUGGE, E. *Introdução ao MongoDB*. São Paulo: Novatec, 2015.

KHETRAPAL, A.; GANESH, V. HBase and Hypertable for large scale distributed storage systems. *[S. n.]*, *[S. l.]*, 2008. Disponível em: https://pdfs.semanticscholar.org/38e2/6a2682f14663a54a75c8af9a6cd350d4c1c1.pdf?_ga=2.85776923.381094989.1593266258-971234948.1587415269. Acesso em: 27 jun. 2020.

KOKAY, M. C. Banco de dados NoSQL: um novo paradigma. *Revista SQL Magazine* 102, Rio de Janeiro, 2012. Disponível em: <http://www.devmedia.com.br/banco-de-dados-nosql-um-novoparadigma-revista-sql-magazine-102/25918>. Acesso em: 27 jun 2020;

KUZNETSOV, S. D.; POSKONIN, A. V. NoSQL data management systems. *Programming and Computer Software*, Moscow, v. 40, n. 6, p. 323-332, 2014.

LEAVITT, N. Will NoSQL databases live up to their promise? *Computer*, New York, v. 43, n. 2, p. 12-14, 2010.

PRAKASH, R.; SINGHAL, M. Dependency sequences and hierarchical clocks: efficient alternatives to vector clocks for mobile computing systems. *Wireless Networks*, Berlin, v. 3, n. 5, p. 349-360, 1997.

PRITCHETT, D. Base: an acid alternative. *ACM Queue*, New York, v. 6, n. 3, p. 48-55, 2008.

SADALAGE, P.; FOWLER, M. *NoSQL essencial: um guia conciso para o mundo emergente da persistência*. São Paulo: Novatec, 2019.

SHAPIRO, M. *et al.* Conflict-free replicated data types. In: DÉFAGO X.; PETIT, F.; VILLAIN, V. (Eds.). *Stabilization, safety, and security of distributed systems: lecture notes in computer science*, v. 6976. Berlin: Springer, 2011. p. 386-400.

STRAUCH, C. *NoSQL databases*. [S. l.: s. n.]: 2011. Disponível em: <https://www.christof-strauch.de/nosql dbs.pdf>. Acesso em: 27 jun. 2020.

SUNEJA, N. Scylladb optimizes database architecture to maximize hardware performance. *IEEE Software*, New York, v. 36, n. 4, p. 96-100, 2019.

TIWARI, S. *Professional NoSQL*. [S. l.]: Wrox, 2011.

YANG, F. *et al.* Druid: a real-time analytical data store. In: ACM SIGMOD INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA, 2014, Snowbird. *Anais...* New York: Association for Computing Machinery, 2014. p. 157-168.

ZVAREVASHE, K.; GOTORA, T. T. A random walk through the dark side of NoSQL databases in big data analytics. *International Journal of Science and Research*, Raipur, v. 3, n. 6, p. 506-509, 2014.

Leituras recomendadas

FOWLER, A. *NoSQL for Dummies*. New Jersey: Wiley, 2015.

PERKINS, L.; REDMOND, E.; WILSON, J. Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. 2. ed. [S. l.]: Pragmatic Bookshelf, 2018.



Fique atento

Os links para sites da web fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais links.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Dica do professor

Na medida em que o volume de dados processados e armazenados nas empresas foi crescendo, também cresceu a necessidade de bancos de dados capazes de comportar essa demanda. Foi aí que surgiu a necessidade de criar bancos de dados distribuídos, capazes de armazenar e processar quantidades enormes de dados. Entretanto, a distribuição dos dados também trouxe algumas limitações.

Na Dica do Professor, você conhecerá o Teorema de CAP e aprenderá quais são essas limitações e como elas influenciam a escolha do banco de dados mais adequado para cada situação.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.



Saiba mais

Para ampliar o seu conhecimento a respeito desse assunto, veja abaixo as sugestões do professor:

Introdução ao NoSQL

Assista neste vídeo uma introdução aos conceitos de bancos de dados NoSQL e suas características.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

History repeats itself: sensible and nonsenSQL aspects of the NoSQL hoopla (em inglês)

Este artigo da ACM conta a história dos bancos de dados NoSQL, o contexto no qual eles surgiram, as diferenças e semelhanças com relação aos bancos de dados relacionais e algumas críticas construtivas.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

A survey on NoSQL stores (em inglês)

Veja neste artigo do tipo survey uma descrição completa sobre a história e evolução dos bancos de dados NoSQL, suas características e decisões técnicas tomadas, além dos desafios que bancos de dados NoSQL têm para o futuro.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.